

```
struct Square {
    Doub operator()(const Doub x) {
        return x*x;
    }
};
```

To use this with a quadrature or other routine, you declare an instance of `Square`

```
Square g;
```

and pass `g` to the routine. Inside the quadrature routine, an invocation of `g(x)` returns the function value in the usual way.

In the above example, there's no point in using a functor instead of a plain function. But suppose you have a parameter in the problem, for example, $f(x) = cx^p$, where c and p are to be communicated from somewhere else in your program. You can set the parameters via a constructor:

```
struct Contimespow {
    Doub c,p;
    Contimespow(const Doub cc, const Doub pp) : c(cc), p(pp) {}
    Doub operator()(const Doub x) {
        return c*pow(x,p);
    }
};
```

In the calling program, you might declare the instance of `Contimespow` by

```
Contimespow h(4.,0.5);           Communicate c and p to the functor.
```

and later pass `h` to the routine. Clearly you can make the functor much more complicated. For example, it can contain other helper functions to aid in the calculation of the function value.

So should we implement all our routines to accept only functors and not functions? Luckily, we don't have to decide. We can write the routines so they can accept *either* a function or a functor. A routine accepting only a function to be integrated from a to b might be declared as

```
Doub someQuadrature(Doub func(const Doub), const Doub a, const Doub b);
```

To allow it to accept either functions or functors, we instead make it a *templated* function:

```
template <class T>
Doub someQuadrature(T &func, const Doub a, const Doub b);
```

Now the compiler figures out whether you are calling `someQuadrature` with a function or a functor and generates the appropriate code. If you call the routine in one place in your program with a function and in another with a functor, the compiler will handle that too.

We will use this capability to pass functors as arguments in many different places in the book where function arguments are required. There is a tremendous gain in flexibility and ease of use.

As a convention, when we write `Ftor`, we mean a functor like `Square` or `Contimespow` above; when we write `fbare`, we mean a “bare” function like `f` above; and when we write `ftor` (all in lower case), we mean an instantiation of a functor, that is, something declared like

```
Ftor ftor(...);           Replace the dots by your parameters, if any.
```

Of course your names for functors and their instantiations will be different.

Slightly more complicated syntax is involved in passing a function to an *object*

that is templated to accept either a function or functor. So if the object is

```
template <class T>
struct SomeStruct {
    SomeStruct(T &func, ...); constructor
    ...
}
```

we would instantiate it with a functor like this:

```
Ftor ftor;
SomeStruct<Ftor> s(ftor, ...
```

but with a function like this:

```
SomeStruct<Doub (const Doub)> s(fbare, ...
```

In this example, fbare takes a single const Doub argument and returns a Doub. You must use the arguments and return type for your specific case, of course.

1.3.4 Inheritance

Objects can be defined as deriving from other, already defined, objects. In such *inheritance*, the “parent” class is called a *base class*, while the “child” class is called a *derived class*. A derived class has all the methods and stored state of its base class, plus it can add any new ones.

“Is-a” Relationships. The most straightforward use of inheritance is to describe so-called *is-a* relationships. OOP texts are full of examples where the base class is ZooAnimal and a derived class is Lion. In other words, Lion “is-a” ZooAnimal. The base class has methods common to all ZooAnimals, for example eat() and sleep(), while the derived class extends the base class with additional methods specific to Lion, for example roar() and eat_visitor().

In this book we use is-a inheritance less often than you might expect. Except in some highly stylized situations, like optimized matrix classes (“triangular matrix is-a matrix”), we find that the diversity of tasks in scientific computing does not lend itself to strict is-a hierarchies. There are exceptions, however. For example, in Chapter 7, we define an object Ran with methods for returning uniform random deviates of various types (e.g., Int or Doub). Later in the chapter, we define objects for returning other kinds of random deviates, for example normal or binomial. These are defined as derived classes of Ran, for example,

```
struct Binomialdev : Ran {};
```

so that they can share the machinery already in Ran. This is a true is-a relationship, because “binomial deviate is-a random deviate.”

Another example occurs in Chapter 13, where objects Daub4, Daub4i, and Daubs are all derived from the Wavelet base class. Here Wavelet is an *abstract base class* or ABC [1,4] that has no content of its own. Rather, it merely specifies interfaces for all the methods that any Wavelet is required to have. The relationship is nevertheless is-a: “Daub4 is-a Wavelet”.

“Prerequisite” Relationships. Not for any dogmatic reason, but simply because it is convenient, we frequently use inheritance to pass on to an object a set of methods that it needs as prerequisites. This is especially true when the same set of prerequisites is used by more than one object. In this use of inheritance, the base class has no particular ZooAnimal unity; it may be a grab-bag. There is not a logical is-a relationship between the base and derived classes.

An example in Chapter 10 is Bracketmethod, which is a base class for several